

Latent World Modeling for Deformable Object Manipulation

Nikunj Parasar Ethan Lai Khush Lalchandani
Stanford University

{nikunj, ethanlai, khushl}@stanford.edu

Abstract

Joint-Embedding Predictive Architectures (JEPAs) offer a compact alternative to pixel-space world models, but their use in deformable-object manipulation remains largely unexplored. In this work, we adapt a LeWorldModel-style action-conditioned JEPA to SoftGym RopeFlatten, where the model predicts rope dynamics from RGB observations, two-picker actions, and proprioceptive state. The model is trained from offline trajectories using next-latent prediction and SIGReg regularization, avoiding pixel reconstruction while preserving an explicit dynamics model for latent-space planning.

We collect a 19,031-episode mixed RopeFlatten dataset spanning expert demonstrations, geometric solver rollouts, random exploration, and manipulative heuristic trajectories, then use the learned world model inside a CEM-based MPC planner. LeWM-MPC reaches a 60% success rate, outperforming ACT, SmolVLA, π_0 LoRA, and prior SoftGym image-based baselines while using only 18.6M parameters. We further analyze training ablations, latent visualizations, a harder Rope U-shape task, and violation-of-expectation tests across simulated rope tasks and real PokeFlex Sponge data. These results suggest that compact JEPA world models can learn useful predictive representations for contact-rich deformable manipulation while remaining small enough for closed-loop planning.

1. Introduction

Manipulation of deformable objects—ropes, cloth, cables, tissue—is essential in robotics, from surgical assistance to garment folding, yet it still remains a fundamentally open problem. Unlike rigid bodies, deformable objects possess high-dimensional configuration spaces, undergo severe self-occlusions, and exhibit chaotic contact dynamics that are difficult to model analytically and very expensive to simulate. [10] Classical model-based planning that relies on finite element method (FEM) simulation is accurate but computationally infeasible for real-time control. On the other end, recent vision-language-action (VLA) models

such as π_0 [6] and OpenVLA [19] plan effectively across manipulation tasks, but carry billions of parameters, creating an inference bottleneck that prevents deployment in real-time closed-loop control.

Latent world models avoid this tradeoff by predicting the future in a compact representation space instead of pixel space. This makes them cheaper than full video prediction while still preserving enough geometric information for planning. Joint-Embedding Predictive Architectures (JEPAs) [2] are a particularly attractive instantiation of this idea. JEPAs avoid the optimization challenges of generative objectives and produce geometrically structured latent spaces by predicting future representations without requiring pixel reconstruction. The recently proposed LeWorldModel (LeWM) [24] demonstrates that a ~ 15 M-parameter end-to-end JEPA world model, stabilized by Sketched Isotropic Gaussian Regularization (SIGReg) [3], plans up to $48\times$ faster than foundation-model-based world models while remaining competitive on 2D and 3D rigid-body control tasks.

This project investigates whether a compact latent world model can learn from pixels sufficiently well to support planning in deformable-object manipulation. We train and evaluate a LeWM-style action-conditioned JEPA on SoftGym [21], a simulation benchmark designed specifically for deformable-object manipulation. The model predicts the latent representation of the next observation from RGB input and two-picker actions. A Cross-Entropy Method (CEM) [8] model-predictive-control planner then uses these latent predictions to choose actions that move the rope toward a goal image.

This setting tests whether the advantages of JEPA-style world modeling extend beyond the rigid-body domains where LeWM has previously been evaluated. We compare this latent-planning approach with direct-policy baselines that learn observation-to-action mappings without an explicit dynamics model. Overall, our goal is to evaluate whether representation-space prediction can serve as a practical middle ground for deformable manipulation.

The input to our model is a short history of top-down RGB rope observations, two-picker robot actions, and the

end-effector state. The model predicts the latent representation of the next observation. At test time, the controller receives the current observation history and a goal image, then outputs the next 8-dimensional two-picker action using latent space MPC.

2. Related Work

Our work builds on many threads: deformable-object manipulation benchmarks, self-supervised representation learning, latent world models for planning, and direct-policy robot-learning baselines.

2.1. Deformable Object Manipulation

Deformable manipulation has been approached through state-estimation pipelines, model-free RL, and model-based control. Preiss *et al.* [26] showed that RNN-based low-dimensional latent dynamics models can achieve high-speed trajectory tracking of deformable objects, but their approach learns from tracked points and measured system outputs rather than RGB, and requires real-world robot interaction for data collection and controller tuning. SoftGym [21] introduced a suite of simulated benchmarks for deformable manipulation with a standard Gym API, including StraightenRope, cloth flattening and folding, and liquid pouring. It also benchmarked several classes of methods, including privileged oracles with access to ground-truth particle information, SAC-based full-state and reduced-state policies [12], and image-based methods that operate from pixels.

Among these image-based baselines are PlaNet [15], a reconstruction-based latent world model with CEM planning, as well as CURL-SAC [29] and DrQ [20]. These SoftGym baselines provide a reference point for evaluating our method on RopeFlatten. A consistent finding is that the image-based methods substantially underperform the state-based oracles, and on StraightenRope in particular the authors report that pixel methods remain far below the oracle ceiling even with extended training.

2.2. JEPA and Latent World Models

Our model builds on the JEPA family of self-supervised methods, which learn representations by predicting the embedding of a target from a context rather than by reconstructing pixels. I-JEPA [2] established this for images and V-JEPA [4] for video. Its action-conditioned successor, V-JEPA 2 [1], showed that video JEPAs can act as world models for planning. Predictive SSL is prone to representation collapse. Previous methods prevent it with heuristics such as EMA teachers, stop-gradients, or variance penalties such as VICReg [5]. LeJEPA [3] instead introduces Sketched Isotropic Gaussian Regularization (SIGReg), a single-hyperparameter term that pushes embeddings toward

an isotropic Gaussian, which is the anti-collapse mechanism this project inherits.

For control, latent world models predict future states in a learned representation space and use those predictions for planning. Early reconstruction-based models, from the original *World Models* [11] through PlaNet [14], Dreamer [13, 16], and TD-MPC2 [17], learn latent dynamics together with pixel or reward decoders. These models have been highly successful, but reconstruction objectives are expensive and vulnerable to compounding prediction error over long rollouts [18].

Representation-space models avoid explicit pixel reconstruction by predicting in feature space. DINO-WM [32] plans over frozen pretrained features, while PLDM [28] and V-JEPA 2-AC [1] train JEPA-style dynamics models for zero-shot, goal-conditioned planning. LeWM [24] brings these ideas together in a compact reconstruction-free and reward-free JEPA world model, using a 192-dimensional token per frame, SIGReg regularization, and CEM planning. Our work adapts the LeWM architecture and training recipe to deformable manipulation in SoftGym [21].

2.3. Direct Policies and Vision-Language-Action Models

A parallel line of work skips explicit dynamics modeling and maps observations directly to actions. The Action Chunking Transformer (ACT) [31] mitigates compounding error in behavior cloning by predicting temporally extended action chunks, and remains a standard imitation baseline.

More recently, vision-language-action (VLA) models have emerged as general-purpose robot policies. These models couple pretrained vision-language representations with action decoders and are trained on large, diverse robot datasets, giving them strong cross-task and cross-embodiment transfer. OpenVLA [19] is a 7B-parameter open VLA trained on roughly 970k demonstrations, while π_0 [6] pairs a vision-language backbone with a flow-matching action expert. Other approaches, including Octo [30] and the compact SmolVLA [27] predict actions without planning over a learned dynamics model.

3. Environment and Data Collection

We study deformable manipulation in SoftGym, which provides a controlled and reproducible benchmark for objects with deformable dynamics. We focus our efforts on the rope manipulation environment as a test for our methods. It is especially useful for our setting as the object has a high-dimensional particle state, severe self-occlusion from a single camera view, and contact-rich bi-manual interactions, while providing clear geometric success metrics. This section describes the `RopeFlatten` environment, our trajectory collection methods, and the diversity of the final data set.

3.1. Rope Simulation Environment

We evaluate in SoftGym’s `RopeFlatten` environment [21], which simulates a deformable rope in NVIDIA FleX [23] and exposes the system through an API. The rope is represented internally as a chain of particles, and control is applied through two provided spherical end-effectors. At each environment step, the agent produces an 8-dimensional action $a_t = [\mathbf{v}_1, g_1, \mathbf{v}_2, g_2]$ where $\mathbf{v}_i \in \mathbb{R}^3$ represents the Cartesian displacements $[\Delta x, \Delta y, \Delta z]$ for each end-effector and $g_i \in \{0, 1\}$ is a binary grasp command.

The SoftGym environment processes 8 physics steps per control decision before returning the next observation. To align the temporal discretization of the dataset with the agent-environment interactions, we record one frame per control decision rather than per physics tick. A single collected episode contains 75 such control decisions and observations, plus the initial frame.

A top-down 128×128 RGB rendering of the simulation is used to record visual observations. For diagnostics and auxiliary supervision, each frame also includes a compact 15-dimensional privileged state s_t consisting of the two end-effector positions and grasp states, the rope center of mass, and the rope bounding-box minimum and maximum. The full particle state is not used as a model input.

3.2. Dataset Construction

We train the world model solely from offline trajectories, without reward annotations or task labels, following the reward-free offline setting used in recent JEPA-based world models [24, 32, 28]. In this setting, we found that trajectory quality strongly affects the model’s ability to learn and predict latent representations, which in turn has a significant impact on downstream planning performance.

We assemble `RopeFlatten` task data from four policies with complementary roles (Table 1): a near-optimal oracle planner that demonstrates the task, and three low-cost exploratory or heuristic policies that broaden state–action coverage. All four are concatenated into a single 19,031-episode set. We train on all 19k collected episodes without a separate validation split, monitor training using prediction loss and embedding statistics, and evaluate success on 50 SoftGym episodes with unseen initial rope configurations. An episode is successful if final normalized performance exceeds 0.8. The following subsections explain why coverage matters alongside task performance, and describe the collection algorithm behind each source.

3.2.1 Expert Trajectories

The expert source supplies near-optimal demonstrations through *oracle* Model-Predictive Control via the Cross-Entropy Method (CEM) while having full observability of

Policy	Episodes	Total Frames	Mean final perf.
CEM Expert	4,031	306K	0.994
Random	5,000	380K	0.409
Geometric Solver	5,000	380K	0.905
Manipulative Heuristic	5,000	380K	0.448
Full set	19,031	1.45M	0.674

Table 1. Summary of the trajectories collected in the project.

the real simulator dynamics.

At each control step, CEM optimizes an action sequence of horizon $H=15$ by sampling a population of 20 candidate sequences from a Gaussian, simulating each candidate, scoring it by the SoftGym straightening reward, and refitting the Gaussian to the top-10% candidates; after 3 such iterations, it executes the first action and replans in receding-horizon fashion. The planner has privileged access to true dynamics, so its trajectories are near-optimal, providing the high-quality pseudo-expert data that offline world-model settings call for [24].

We generate these trajectories with the Dynamics Oracle CEM controller from the SoftAgent benchmark [21] on `RopeFlatten`, then re-execute the planned actions in the rendering environment to obtain the image observations the model trains on. Since every CEM evaluation step requires full physics simulation, collection is bound by CPU throughput while hosting the FleX engine on a supported GPU; we therefore parallelized the collection task across 3 nodes with a total of 24 GPUs and 144 vCPUs to host concurrent simulator rollouts. Due to the computationally intensive nature of this algorithm our final expert episode count was ultimately limited by available compute hours. The resulting 4,031-episode set has a mean final normalized performance 0.994 and a mean peak performance of 0.998 as defined in section 5.1.

3.2.2 Random and Heuristic Trajectories

The remaining three sources are cheap to collect and deliberately sub-optimal; their purpose is to populate the regions of state–action space that the full dynamics oracle avoids. **Random** rollouts sample picker displacements uniformly from the action bounds. These episodes almost never result in a straightened rope (mean final performance 0.409), but they serve as a source of collisions, missed grasps, and misaligned motions, elements that a planner must be able to predict when it scores imperfect candidate actions. The **geometric** policy is a scripted state machine that reads the rope endpoints from the simulator state, moves a picker above each endpoint, descends, closes the grippers, lifts, stretches the rope along its principal axis, and holds the final pose; it yields clean, smoothly actuated task completions (0.905). The **manipulative** policy instead grasps randomly chosen *interior* rope particles and follows smooth random waypoints, generating curls, folds, and partially lifted con-

figurations (0.448) that neither the endpoint-focused expert nor the contact-sparse random policy produces. Together, these episodes complement the expert trajectories to provide broad coverage of intermediate rope geometries.

4. Method: Latent World Modeling

Our method builds off the LeWorldModel (LeWM) recipe [24], learning a compact, action-conditioned latent dynamics model from offline video, then using that model for goal-conditioned planning. We chose this architecture because it provides a compact reconstruction-free dynamics model while preserving an explicit planning interface through latent rollouts.

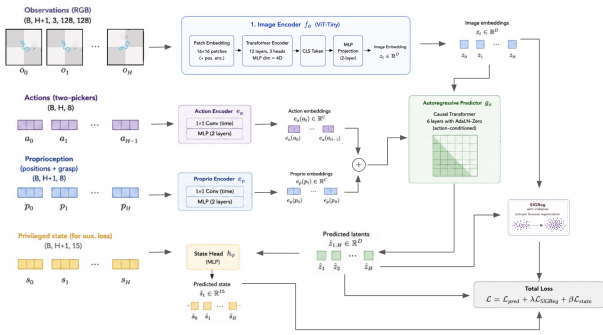


Figure 1. LeWorldModel Architecture for latent-MPC planning.

4.1. World-model Architecture

Let o_t denote the RGB observation, $a_t \in \mathbb{R}^8$ the two-picker action, and $p_t \in \mathbb{R}^8$ the proprioceptive vector (end-effector positions and grasp flags). A shared image encoder f_θ maps each frame to a low-dimensional latent

$$z_t = f_\theta(o_t) \in \mathbb{R}^D$$

f_θ is a compact ViT-Tiny [9] inspired encoder, with each 128×128 image split into 16×16 patches (64 patch tokens plus a [CLS] token) and processed by a 12-layer, 3-head transformer with a 4D MLP. We take the final [CLS] token as the frame representation and project it through a two-layer MLP before the prediction loss. The full model has $\approx 18.6\text{M}$ parameters, allowing us to train end-to-end on a single GPU.

The dynamics of the rope are modeled by an autoregressive transformer predictor g_ϕ that, from a history of $H = 3$ latents and their conditioning vectors, predicts the next latent:

$$\hat{z}_{t+1} = g_\phi(z_{\leq t}, c_{\leq t}).$$

g_ϕ stacks 6 causal transformer blocks with adaptive LayerNorm-zero (AdaLN) conditioning [25]: the conditioning vector produces per-block shift, scale, and gate parameters for the attention and MLP branches, making the

transition explicitly action-conditioned while using a single latent token per frame.

The robot actions and proprioceptive data are input using two lightweight encoders—each a 1×1 temporal convolution followed by a two-layer MLP—and summed into the conditioning vector $c_t = e_a(a_t) + e_p(p_t)$.

4.1.1 SIGReg

Following the LeWorldModel architecture [24], we use Sketched Isotropic Gaussian Regularization (SIGReg) to prevent collapse in the JEPA latent space. Since our model predicts future embeddings rather than reconstructing pixels, the encoder could otherwise learn latents that are easy to predict but not useful for planning. SIGReg avoids this by encouraging the batch of projected image latents to match an isotropic Gaussian distribution, keeping the representation spread out and well scaled.

In practice, SIGReg projects latents onto many random one-dimensional directions and compares each projected distribution to a standard Gaussian using a sketched characteristic-function distance:

$$\mathcal{L}_{\text{SIGReg}} = \frac{1}{M} \sum_{m=1}^M d_{\text{CF}}(\{u_m^\top z_i\}_{i=1}^N, \mathcal{N}(0, 1)).$$

We use $M = 1024$ random projections, $K = 17$ frequency samples, and weight this term by $\lambda = 0.09$. This is important for latent-MPC because planning compares predicted latents to goal latents; if the representation collapsed or had poor scale, latent distances would no longer correspond reliably to differences in rope configuration.

4.1.2 Fusion and Encoder Guidance

For rope manipulation, we fuse proprioception into the dynamics-conditioning for the prediction transformer, and not into the goal-matched latent z_t . This keeps z_t focused on the rope geometry, so that planning-time latent distances compare rope configurations rather than the resulting end-effector poses. This is especially relevant in SoftGym, where the default end-effectors are visually similar to the background and are hard to infer from images alone. Putting end-effector state directly into z_t would make latent goal matching penalize irrelevant differences in gripper location.

We also add an auxiliary compact-state head for *geometric guidance*. A next-latent prediction objective would only require the latent to be useful for short-horizon prediction, so the [CLS] representation could ignore important rope geometry that matters for planning, even when regularized via SIGReg. Let $s_t \in \mathbb{R}^{15}$ denote the compact state logged in the dataset: two picker positions, rope center of mass, and rope bounding-box minimum and maximum. During

training, a small MLP h_ψ predicts this state from the image latent $\hat{s}_t = h_\psi(z_t)$. This produces an auxiliary regression loss on every frame in the sampled window

$$\mathcal{L}_{\text{state}} = \frac{1}{B(H+1)} \sum_{b=1}^B \sum_{t=0}^H \|\hat{s}_{b,t} - s_{b,t}\|_2^2$$

h_ψ is not used at inference time, since the planner for task solving will not have access to privileged particle information. Its purpose is to guide the learned latent during training: z_t should remain a goal-matching image representation, but it should also preserve the rope’s extent, bounding box, and coarse spatial layout.

4.1.3 Training Objective

We train on contiguous windows sampled from individual trajectories. Within each sampled window we renumber time locally as $0, \dots, H$. Thus a batch contains B sequences of observations $(o_{b,0}, \dots, o_{b,H})$, actions $(a_{b,0}, \dots, a_{b,H-1})$, and proprioceptive states $(p_{b,0}, \dots, p_{b,H})$.

The image encoder produces target latents $z_{b,t} = f_\theta(o_{b,t}) \in \mathbb{R}^D$ for all $H+1$ frames. For each context time $t \in \{0, \dots, H-1\}$, the predictor receives the history of image latents up to t together with the action/proprioception conditioning $c_{b,t} = e_a(a_{b,t}) + e_p(p_{b,t})$, and outputs a one-step prediction $\hat{z}_{b,t+1} = g_\phi(z_{b,0:t}, c_{b,0:t})$. The primary training objective averages the mean-squared errors of the *latent* predictions over the batch and the H predicted transitions

$$\mathcal{L}_{\text{pred}} = \frac{1}{BH} \sum_{b=1}^B \sum_{t=0}^{H-1} \|\hat{z}_{b,t+1} - z_{b,t+1}\|_2^2$$

where $z_{b,t+1}$ is the encoder latent of the actual next frame and $\hat{z}_{b,t+1}$ is the predictor’s imagined next latent. The full objective is

$$\mathcal{L} = \mathcal{L}_{\text{pred}} + \lambda \mathcal{L}_{\text{SIGReg}} + \beta \mathcal{L}_{\text{state}},$$

where $\mathcal{L}_{\text{state}}$ is the auxiliary state-regression loss mentioned in section 4.1.2.

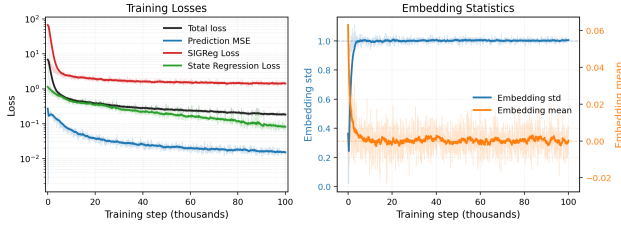


Figure 2. Training curves for the LeWM model

We use $\lambda = 0.09$, and $\beta = 0.5$. We optimize with AdamW [22] (learning rate 5×10^{-5} , weight decay 10^{-3} ,

gradient clipping 1.0), batch size 128, and bfloat16 mixed precision; actions, proprioception, and state are z-score normalized using training-set statistics. The model was trained for 100,000 steps on a single NVIDIA H100 GPU. As seen in Figure 2, our model achieves a training error of 0.0152 while SIGReg keeps the latent embeddings $\sim \mathcal{N}(0, 1)$ preventing collapse. We saw no signs of overfitting, and because we evaluate on unseen configurations, task success reflects generalization rather than memorization.

4.2. Planning in Latent Space

4.2.1 Planning Objective

At test time, the world model is frozen and used as the dynamics model for Model-Predictive Control. A single goal image o_g is encoded once to obtain the goal latent $z_g = f_\theta(o_g)$. For example, in RopeFlatten tasks, o_g is selected from the “flattest” frame in a scripted expert reference trajectory.

Each replan begins by encoding the most recent H real observations into a history $(z_{t-H+1}, \dots, z_t)$ and retaining the previous $H-1$ real actions. We define a candidate plan $A_{0:K-1}$ as a sequence of future raw actions $(a_t, \dots, a_{t+K-1})$ where K is the planning horizon.

To score a candidate plan, we must form the same temporal alignment used during training to predict using the forward pass of our model. To do this, we concatenate the $H-1$ past actions with the K proposed future actions, and use A_0 to immediately predict the next latent. The predictor is then rolled forward autoregressively with a sliding window of size H , producing imagined latents $\hat{z}_{t+1}, \dots, \hat{z}_{t+K}$. The per-step planning objective is to minimize the fixed-goal terminal cost:

$$J(A_{0:K-1}) = \|\hat{z}_{t+K} - z_g\|_2^2.$$

As a result, the planner does not need to optimize in pixel space or use the ground truth simulator reward, since it can evaluate actions by the distance between an imagined terminal latent and the encoded goal.

4.2.2 Planning Solver

We solve the above finite-horizon optimal-control problem employing the Cross-Entropy Method in our MPC planner. At each CEM iteration, we sample 300 candidate plans in the normalized action space, evaluate their terminal *latent* costs, keep the 30 lowest-cost plans, and refit the Gaussian sampling distribution to those plans. We run 30 CEM iterations with planning horizon $K = 5$, corresponding to a total 45,000 imagined model steps to evaluate 9,000 candidate action sequences.

To be consistent with our prediction transformer’s model architecture, proprioceptive conditioning is also rolled for-

ward consistently. For each candidate plan, future end-effector positions are obtained by numerically integrating the planned displacement components of $\mathbf{v}_i$, and the grasp bit-states are derived from the planned grip actions. These imagined proprioceptive states are then normalized and embedded by e_p before being added to the action embeddings so that every imagined transition is conditioned on both the proposed action and the implied robot state, matching the proprioceptive fusion used during training.

CEM optimizes the normalized action variables $u_i = (a_i - \mu_a)/\sigma_a$ where μ_a and σ_a are action mean and standard deviations from the training set to match the predictor’s expected input scale. The optimal normalized actions are un-normalized and clamped to the SoftGym environment action bounds before execution. We execute the planned segment and then re-encode the true environment response before replanning to mitigate the accumulation of open-loop prediction error.

5. Experiments

5.1. Task Performance and Baseline

SoftGym measures progress in `RopeFlatten` by the distance between the two rope endpoints. Let d_t be the endpoint distance at time t and L the total rope length. The task reward is $r_t = -|d_t - L|$, which is maximized when the rope is fully straightened. We report the corresponding normalized performance:

$$p_t = \frac{r_t - r_{\text{init}}}{r_{\text{max}} - r_{\text{init}}}, \quad r_{\text{max}} = 0, \quad (1)$$

such that $p_t = 1$ represents a fully straightened rope and values near 0 correspond to fully coiled configurations.

We compare against two classes of baselines. The first are the image based reinforcement learning methods in SoftGym’s SoftAgent: CURL-SAC, DrQ, and PlaNet, which represent prior state of the art for pixel-based control on this task. The second are direct policies that learn observation-to-action mappings without an explicit dynamics model or planning step. ACT [31] ($\sim 52\text{M}$) is an action-chunking imitation policy. SmolVLA [27] ($\sim 450\text{M}$) is a compact pretrained VLA built on LeRobot [7]. π_0 [6] ($\sim 3\text{B}$), a flow matching VLA, is our heavyweight reference for measuring how well priors from large-scale real-robot data transfer to deformable simulation.

We train all three on the CEM expert subset rather than the full mixed set used for our model. Since imitation and VLA fine-tuning regress toward demonstrated behavior, expert-only is the strongest setting for each baseline, which we verified empirically. We convert these trajectories to the LeRobot format and evaluate every method on identical configurations, goal images, success threshold, and hardware. Each baseline was trained past convergence and evaluated at its best checkpoint.

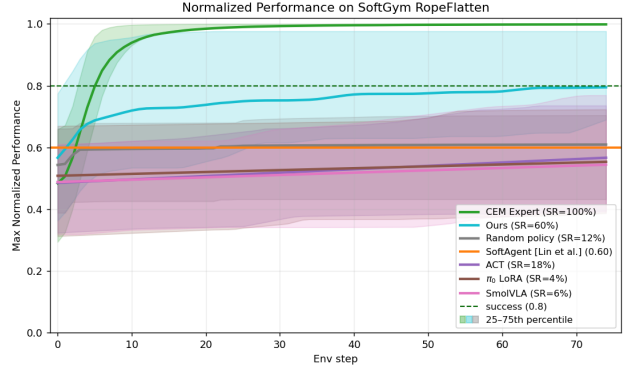


Figure 3. Benchmarked algorithms versus our latent-MPC ($n=50$).

As shown in Figure 3, LeWM-MPC achieves a 60% success rate, outperforming ACT (18%), SmolVLA (6%), π_0 LoRA (4%), and SoftGym’s image-based RL baselines by a substantial margin. ACT is the strongest direct policy, which is consistent with its action-chunking design, mitigating compounding error in the bi-manual 8-DoF action space. We hypothesize that the VLAs underperform despite their scale due to their pretraining data consisting predominantly of real-robot manipulation of rigid objects with no analogous deformable-contact behavior. Due to this, LoRA fine-tuning on 4k expert episodes is insufficient to overcome this distribution mismatch.

Policy	Params	Mean final perf.	Avg latency (ms)
Our Model	$\sim 18.6\text{M}$	0.848	821.6
ACT	$\sim 52\text{M}$	0.542	25.7
SmolVLA	$\sim 450\text{M}$	0.444	282.6
π_0 LoRA	$\sim 3\text{B}$	0.432	308.9

Table 2. Mean final performance and per-step planning latency.

Table 2 reports per-step planning latency averaged over 50 episodes. ACT is fastest at 25.7 ms, given its single-pass chunked design, but operates open loop within each chunk. SmolVLA (282.6 ms) and π_0 LoRA (308.9 ms) incur autoregressive and flow-matching decoding costs that scale with parameter count, making high-frequency closed-loop control difficult in this setting. LeWM-MPC takes 821.6 ms per replan, but this covers evaluating 9,000 candidate action sequences across 30 CEM iterations entirely in the 192-dimensional latent space. Our model achieves higher success rates than the VLAs at a fraction of their size, while still replanning fast enough for closed-loop control.

5.2. Latent Visualizations

To qualitatively inspect what information is preserved in the learned representation, we train a lightweight decoder from frozen LeWM latents back to RGB observations. The decoder is used only for visualization and is not part of the world model training objective or the MPC. This diagnostic

helps us determine whether the 192-dimensional latent contains enough spatial information about the rope to support goal-conditioned planning.

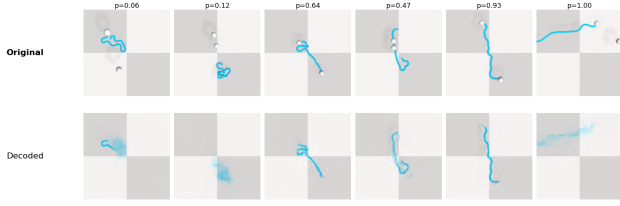


Figure 4. Decoded visualizations from frozen LeWM latents in $\mathbb{R}^{192}$.

Figure 4 shows that the learned latent representation captures the global configuration of the rope. The decoded outputs preserve the most significant information to the downstream encoder, including rope extent, curvature, and end-point layout. This supports the use of latent distance as a planning cost.

The diagnostic decoder uses a cross-attention transformer design faithfully reproducing the *LeWorldModel* appendix: Using $8 \times 8 = 64$ learnable per-patch query tokens (one per 16×16 output patch) cross-attending to the projected latent $\hat{z} = \text{proj}(f_{\theta}(o))$. Each query is decoded by a linear head to a $16 \times 16 \times 3$ pixel patch. The resulting 128×128 reconstruction is supervised with pixel MSE against the original frame. Each decoder is trained for 8,000 AdamW steps with a learning rate of 2×10^{-4} , and batch size of 64. The entire procedure takes under five minutes on an NVIDIA H100 and was used as a qualitative sanity check of our encoded latents.

5.3. Training Experiments

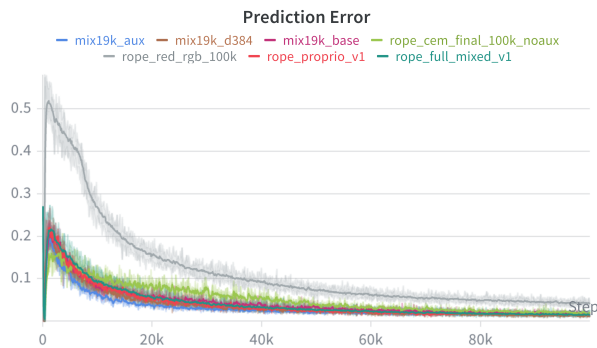


Figure 5. Prediction errors across our various training runs.

The relatively cheap training cost of our model allowed us to iterate empirically across data composition, visual representation, model capacity, and geometric supervision, finding that prediction MSE was an unreliable proxy for task success. Models with similar MSEs could differ by

20%+ in success rate (Table 3). Starting from a diverse 15k mixed dataset (25% SR), rendering pickers red to aid visual end-effector state actively hurt (12%), and training on CEM-only expert data only achieved 38% due to data coverage gaps. Combining oracle CEM episodes with broad coverage in the 19k mix raised the ceiling while doubling model capacity ($D=384$) gave no benefit (46%), but adding the auxiliary state head, which grounds the latent representation of rope geometry at training time lifted task success rates from 46% to 60%.

Run	Key change vs. final	Dataset	Pred MSE↓	SR%↑
mix19k_aux	Final model	19k mixed	0.0152	60
mix19k_base	–aux state head	19k mixed	0.0168	46
mix19k_d384	+capacity ($D=384$)	19k mixed	0.0121	46
rope_cem_noaux	CEM-only data, –aux	4k CEM only	0.0188	38
rope_proprio_v1	–aux state head	15k (no CEM)	0.0189	30
rope_red_rgb	Red picker visual, –proprio	15k (no CEM)	0.0415	12
rope_full_mixed_v1	Pre-CEM data, –aux, –proprio	15k (no CEM)	0.0134	24

Table 3. Ablations table comparing the most impactful training results that guided our design decisions towards the final world model

5.4. Solving Harder Rope Tasks

To test whether our approach extends beyond straightening, we evaluate LeWM on a harder Rope U-shape task. Unlike RopeFlatten, where maximizing endpoint distance is often aligned with local pulling actions, the U-shape task requires the rope to match a more constrained global geometry. The planner must bend the rope into a curved target while preserving continuity, so actions that improve local straightness can still move the system away from the desired configuration. This makes the task a stronger test of whether the learned latent dynamics support goal-conditioned planning.



Figure 6. Example rollout for the harder Rope U-shape task.

We collected 1000 near-expert U-shape trajectories using a scripted geometric policy and trained a new LeWM model on the original full RopeFlatten dataset plus these U-shape demonstrations. The task uses the same observation, action, and proprioceptive inputs as RopeFlatten, but changes the goal distribution from straight configurations to curved U-shaped targets, as shown in Figure 6. On 10 held-out U-shape rollouts, the model reached a mean final normalized performance of 0.445 ± 0.048 and a mean peak performance of 0.630 ± 0.021 (SEM). Using a 0.5 final-

performance threshold, 3/10 rollouts ended successfully, while all 10 rollouts reached peak performance above 0.5 at some point during execution. These results are weaker than RopeFlatten, but still useful: the model often discovers actions that transiently form the desired U-shape, while the drop from peak to final performance shows that maintaining the curved configuration over longer horizons remains difficult.

5.5. Violation of Expectation

Inspired by the violation-of-expectation evaluation in LeWorldModel [24], we test whether our learned latent dynamics assign greater surprise to futures that are inconsistent with the model’s prediction. For each sequence, LeWM receives the same observation history and action context, predicts a future latent state $\hat{z}_{t+1}$, and is evaluated against the encoded latent z_{t+1} of a target future observation:

$$S_t = \|\hat{z}_{t+1} - z_{t+1}\|_2^2.$$

Because the action sequence is fixed across conditions, differences in S_t measure future-state inconsistency rather than action mismatch.

For RopeFlatten and Rope U-shape, we generate matched SoftGym rollout triplets at evaluation time: three rollouts from the same simulator state with the same action sequence, but different future targets. The *none* rollout is unmodified; the *visual* rollout changes only rendered rope appearance; and the *physically implausible* rollout teleports part of the rope after the perturbation step. We use 10 triplets per SoftGym task and score a short window around the perturbation. We also evaluate PokeFlex Sponge, a real-robot sponge-poking dataset, as an external real-world sanity check rather than a direct baseline against SoftGym. Since real trajectories cannot be re-simulated with controlled teleportations, we use analogous target perturbations: an appearance edit for the visual condition and a mismatched future-state target for the physically implausible condition.

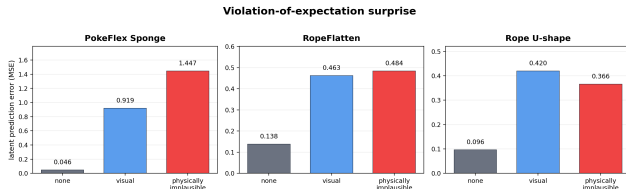


Figure 7. Violation-of-expectation surprise comparisons.

Figure 7 shows that perturbed futures consistently produce greater surprise than unmodified futures. PokeFlex Sponge has the strongest separation, increasing from 0.046 to 0.919 under the visual perturbation and 1.447 under the physically implausible future. RopeFlatten increases from 0.138 to 0.463 and 0.484, with physical surprise slightly

higher than visual surprise. Rope U-shape increases from 0.096 to 0.420 and 0.366; here, visual surprise is slightly higher, likely because the learned latent space is especially sensitive to rendered rope appearance while the teleportation can remain geometrically close to plausible U-shaped configurations over the short scoring window.

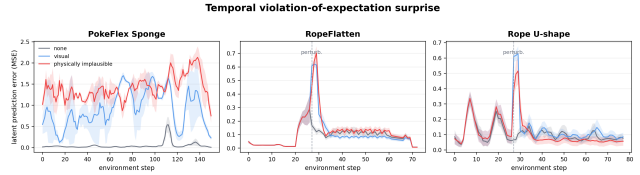


Figure 8. Temporal violation-of-expectation curves.

The temporal curves in Figure 8 show surprise rising near the perturbation step for the SoftGym tasks, then decreasing as later context includes the perturbed observations. This suggests that LeWM responds most strongly when the observed future first diverges from the predicted latent dynamics. Overall, the VoE test provides evidence that the model learns a predictive representation of deformable object state that distinguishes expected futures from visually or physically inconsistent ones.

6. Conclusion

We adapted a LeWorldModel-style [24] action-conditioned JEPA to deformable-object manipulation in SoftGym’s [21] RopeFlatten task and showed that compact latent world modeling extends from rigid-body control to contact-rich rope manipulation. Our model was trained on a 19k-episode mixed dataset with 18.6M parameters and reached a 60% success rate with latent-space MPC, outperforming the SoftGym image-based RL baselines, an action-chunking imitation policy (ACT [31]), and pretrained VLAs (SmoVLA [27], π_0 LoRA [6]) that are up to 150 times larger. Latent decoding and violation-of-expectation tests support that the learned representation captures rope geometry and reacts to physically inconsistent futures. The harder Rope U-shape task suggests that the planner can extend beyond simple endpoint-pulling behavior to more constrained goal geometries, though maintaining these configurations over long horizons remains difficult.

Several limitations point to future work. Compute constraints prevented us from directly comparing our model against fine-tuning a full video JEPA world model such as V-JEPA 2-AC [1], which would be a natural next baseline. Another exciting direction is transferring this recipe to real-robot deformable manipulation with more challenging deformable benchmarks like cloth folding on physical hardware where the latency and parameter advantages of compact latent world models matter most.

7. Contribution and Acknowledgments

7.1. Contribution Statement

All three authors collaborated on the initial problem formulation and read and discussed the relevant literature on JEPA-based world models, deformable-object manipulation benchmarks, and vision-language-action policies to establish the background and motivation for our method.

Nikunj Parasar: Collected the manipulative heuristics trajectories and CEM expert trajectories using the Dynamics Oracle controller and set up the multi-GPU data collection environment. Implemented and trained the LeWM-style world model, conducted training ablations, latent decoder transformer, and integrated the MPC planner with the SoftGym simulation environment. Wrote the Data and Methods section covering the SoftGym environment, world model architecture, and latent space planning.

Ethan Lai: Collected the geometric solver trajectories. Conducted the Rope U-Shape task setup/implementation, trained a PokeFlex Sponge LeWM model for validation experiments, and conducted the Violation-of-Expectation evaluation across RopeFlatten, Rope U-shape, and PokeFlex Sponge. Wrote the Abstract, Violation-of-Expectation and Harder Rope Tasks, and contributed to the dataset/architecture sections of the report.

Khush Lalchandani: Collected the random exploration trajectories for the dataset. Implemented and trained all direct-policy and VLA baselines (ACT, SmolVLA, and π_0 LoRA). Set up the data conversion pipeline and ran the baseline accuracy and latency comparisons against our model. Wrote the Introduction/Related Work and direct policy baseline/latency sections of the report.

Joint Contribution: All authors contributed to experimental analysis, report editing, result synthesis, and poster creation. The team met weekly to discuss progress, divide next steps, and review results.

7.2. Generative AI Usage Statement

We used generative AI tools (Claude and Codex) in a limited assistive capacity for this project. Specifically, the tools were used for:

- Brainstorming and refining the structure of the project/report sections
- Assisting with LaTeX formatting and figure layout
- Helping generate training and evaluation scripts and code completion which was always reviewed and verified by the authors to ensure correctness
- Helping generate figures given data and a specified format provided by the authors.

All technical contributions, including model design, training, evaluation, data collection, and analysis were performed by the authors.

References

- [1] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, Mojtaba, Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, S. Arnaud, A. Gejji, A. Martin, F. R. Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025.
- [2] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023.
- [3] R. Balestriero and Y. LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025.
- [4] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas. Revisiting feature prediction for learning visual representations from video, 2024.
- [5] A. Bardes, J. Ponce, and Y. LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning, 2022.
- [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [7] R. Cadene, S. Aliberts, F. Capuano, M. Aractingi, A. Zouitine, P. Kooijmans, J. Choghari, M. Russi, C. Pascal, S. Palma, M. Shukor, J. Moss, A. Soare, D. Aubakirova, Q. Lhoest, Q. Gallouédec, and T. Wolf. Lerobot: An open-source library for end-to-end robot learning, 2026.
- [8] P. T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein. A tutorial on the cross-entropy method. *Annals of Operations Research*, 134:19–67, 2005.
- [9] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2021.
- [10] F. Gu, Y. Zhou, Z. Wang, S. Jiang, and B. He. A survey on robotic manipulation of deformable objects: Recent advances, open challenges and new frontiers, 2023.
- [11] D. Ha and J. Schmidhuber. World models, 2018.
- [12] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018.
- [13] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination, 2020.
- [14] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2018.
- [15] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels, 2019.

- [16] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models, 2024.
- [17] N. Hansen, H. Su, and X. Wang. Td-mpc2: Scalable, robust world models for continuous control, 2024.
- [18] M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization, 2021.
- [19] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model, 2024.
- [20] I. Kostrikov, D. Yarats, and R. Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels, 2021.
- [21] X. Lin, Y. Wang, J. Olkin, and D. Held. Softgym: Benchmarking deep reinforcement learning for deformable object manipulation, 2021.
- [22] I. Loshchilov and F. Hutter. Decoupled weight decay regularization, 2019.
- [23] M. Macklin, M. Müller, N. Chentanez, and T.-Y. Kim. Unified particle physics for real-time applications. *ACM Transactions on Graphics (TOG)*, 33:1 – 12, 2014.
- [24] L. Maes, Q. L. Lidec, D. Scieur, Y. LeCun, and R. Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026.
- [25] W. Peebles and S. Xie. Scalable diffusion models with transformers, 2023.
- [26] J. A. Preiss, D. Millard, T. Yao, and G. S. Sukhatme. Tracking fast trajectories with a deformable object using a learned model. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 1351–1357, 2022.
- [27] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. Smolvla: A vision-language-action model for affordable and efficient robotics, 2025.
- [28] V. Sobal, W. Zhang, K. Cho, R. Balestriero, T. G. J. Rudner, and Y. LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models, 2025.
- [29] A. Srinivas, M. Laskin, and P. Abbeel. Curl: Contrastive unsupervised representations for reinforcement learning, 2020.
- [30] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy, 2024.
- [31] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware, 2023.
- [32] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning, 2025.